SMART PARKING SYSTEM

DATABASE & DOCKER INTEGRATION TEST REPORT

Document Version: 1.0

Test Date: October 12, 2026

Tester: Hoàng Phan

Project: Smart Parking System – Web MVP

Test Type: Database and Docker Integration Smoke Test

1. Objective

The objective of this test was to verify whether the new database Docker setup could run concurrently with the existing database environment and whether the databases were accessible through their published ports and capable of executing a basic SQL query.

The test focused on environment setup and database connectivity. API endpoint testing and business-function testing were excluded from this scope.

2. Test Environment

- Host environment: Windows with Docker Desktop using Linux containers.
- Database image: PostgreSQL 16-alpine.
- Existing database container: `smartparking_db`.
- New database services: User, Parking, Reservation, Payment, Notification, and IoT.
- Existing database host port: 5433.
- New database host ports: 5441, 5442, 5443, 5444, 5445, and 5446.
- Docker Compose configuration:
    - `compose.schema-integration.yml`
    - `scripts/database/compose.remaining-services.yml`

- Git working branch: `develop`.

3. Test Scope

The following checks were performed:

1. Validate the combined Docker Compose configuration.
2. Start the six new database services together.
3. Verify that the existing database and the six new database containers could run concurrently.
4. Verify TCP connectivity to all seven published database ports.
5. Execute a basic SQL query against each database.
6. Inspect the current User database health status and recent logs.

4. Test Results

Overall Result: PASS for the assigned database and Docker connectivity smoke-test scope.

Test ID	Test Description	Result
DBDOCKER-001	Docker Compose configuration validation	PASS
DBDOCKER-002	Start six new database services	PASS
DBDOCKER-003	Concurrent operation with the existing database	PASS
DBDOCKER-004	TCP connectivity for all seven database ports	PASS
DBDOCKER-005	Basic SQL query against all seven databases	PASS
DBDOCKER-006	Current User database health and recent log inspection	PASS
DBDOCKER-007	Fresh-volume initialization and full schema verification	N/A – Not verified
DBDOCKER-008	Full repository setup script validation	N/A – Not executed

4.1 Container Status

The following database containers were running concurrently during the verification:

- Existing database: `smartparking_db`
- User database: `smartpark-schema-user-db-1`
- Parking database: `smartpark-schema-parking-db-1`

- Reservation database: `smartpark-schema-reservation-db-1`
- Payment database: `smartpark-schema-payment-db-1`
- Notification database: `smartpark-schema-notification-db-1`
- IoT database: `smartpark-schema-iot-db-1`

All six new database containers reported a healthy status after startup.

4.2 Port Connectivity

TCP connectivity was verified for all seven published ports.

Database	Host Port	Result
Existing database	5433	PASS
User	5441	PASS
Parking	5442	PASS
Reservation	5443	PASS
Payment	5444	PASS
Notification	5445	PASS
IoT	5446	PASS

All tested ports returned `TcpConnect=True`.

4.3 Basic SQL Connectivity

A basic `SELECT 1;` query was executed against the existing database and each of the six new databases.

Result: All seven databases accepted the tested connection and returned `1`.

This confirms basic database connectivity and query execution. It does not establish that every expected table, constraint, relationship, or migration is correct.

4.4 Health and Log Inspection

The inspected User database health status was `healthy`, with a failing streak of zero. The inspected recent log window included the message indicating that the database system was ready to accept connections.

Older authentication and role-related errors were visible in earlier log output. No recent fatal issue was confirmed in the inspected 30-minute log window. These older entries were recorded as observations rather than confirmed current failures.

5. Limitations

The following activities were not verified as part of this test:

- Full database schema and table validation against all SQL source scripts.
- Initialization from completely fresh volumes.
- Data persistence after stopping and restarting the database containers.
- End-to-end execution of the repository's full `setup.ps1` workflow.
- Backend API and business-function testing.

The User database logs indicated that an existing database directory was reused and initialization was skipped. Therefore, this run does not prove that all initialization scripts executed from scratch.

6. Conclusion

The database and Docker integration smoke test passed within the assigned scope.

The existing database and all six new database containers ran concurrently. All seven published ports accepted TCP connections, and all seven databases successfully executed the basic `SELECT 1;` query.

No source code or Docker Compose configuration files were modified during this verification.

The result supports the conclusion that the new database Docker setup can run alongside the existing database environment and that basic database connectivity is working. Further schema, fresh-volume initialization, and restart-persistence checks would be required before claiming full database setup validation.

Final Status: PASS – Database/Docker coexistence and basic connectivity verified within the stated scope.